

DELHI AIRCAST — PS-13 AI-POWERED AIR QUALITY FORECASTING

DETAILED PROJECT EXPLORATION REPORT

Prepared: 9 October 2026

Project repository inspected: /root/Delhi-AirCast-PS13

PURPOSE AND EVIDENCE NOTE

This report explains the project's goals, code structure, data pipeline, models, evaluation claims, deployment paths, and limitations based on the checked-out source tree and its documentation. The repository contains reports and experiment notes but does not contain the large raw and processed research datasets described in its inventory; they are intentionally omitted from Git. Consequently, numbers attributed below to experiments are documented project results, not independently recomputed during this inspection. The tracked compact WAQI pilot model binaries and their evaluation.json are present. Existing Word/PDF deliverables are in the repository. The report date is the inspection date; some generated source inventory snapshots and experiment notes are dated earlier.

EXECUTIVE SUMMARY

Delhi AirCast is an academic PS-13 implementation for short-term air-quality forecasting in Delhi. It combines a research pipeline for hourly CPCB/DPCC station measurements with weather, atmospheric-model, fire-event, and spatial features; trains and compares forecasting models; calculates CPCB AQI sub-indices; and exposes forecasts through both a Python API and web dashboard.

There are two materially different forecast experiences:

1. Historical station-level research forecasts: predict PM2.5 concentration (micrograms per cubic metre) at 39 Delhi stations for 1
2. A compact live pilot: fetches WAQI readings for Pusa and R.K. Puram and predicts the WAQI individual PM2.5 index on a US-EPA sca

The project shows meaningful historical model improvements over persistence in reported tests, while documenting that performance varies by horizon and station. The global model had a notable site_113 regression in one-hour results; sequence and graph models did not beat XGBoost in controlled comparisons. The 6-hour multi-pollutant subset experiment improves AQI metrics over a PM2.5-only proxy, but omits SO2 and NH3. The live pilot is limited to two sites and its evaluation is based on historical 2025 periods, not independent present-day validation.

PROJECT PURPOSE AND SCOPE

The original problem statement identifies that residents lack neighbourhood-scale, short-horizon air-quality information for planning outdoor activity. The project goal is to provide station-aware forecasts and an interactive interface. The engineering target is hourly Delhi station forecasts, not a validated continuous map of every neighbourhood. The repository's strategy document emphasizes choosing models from measured backtests, not from model novelty, and treating LSTM, TCN, and graph neural networks as challengers until they outperform a simpler incumbent.

The current product contract in MODEL_STRATEGY.md proposes forecasts at +1, +3, +6, +12, and +24 hours, full hourly paths, and calibrated uncertainty intervals. The code and reports demonstrate direct point forecasts at selected horizons. The README and experiment ledger identify prediction intervals as planned work; they are not part of the demonstrated production outputs described here.

REPOSITORY MAP

- README.md: setup, data acquisition, training, local serving, live pilot, dashboard, and deployment instructions.
- problem.md: short PS-13 problem statement and suggested technologies.
- DATA_SOURCES.md / KAGGLE_DATASETS.md: source roles and provenance policy.
- MODEL_STRATEGY.md: proposed evaluation protocol and model ladder.
- EXPERIMENTS.md: measured experiments, scores, decisions, and caveats.
- scripts/: downloaders, normalizers, dataset builders, evaluation programs, training scripts, and model exporters.
- src/delhi_aircast/aqi.py: AQI breakpoint tables and AQI/proxy calculations.
- src/delhi_aircast/api.py: FastAPI endpoints, offline model loading, WAQI live endpoint, weather lookup, and serving logic.
- dashboard.py: separate local Streamlit research dashboard.
- aircast-web/: Next.js TypeScript dashboard for the web/API deployment.
- models/waqi_live_index/: compact tracked model artifacts and an evaluation summary for the two-site live pilot.
- data/reference/: CPCB reference documents.
- figures/: architecture, station, model comparison, and interface illustrations.
- Delhi_AirCast_PS13_Project_Report.*: detailed report already in the project.
- Delhi_AirCast_PS13_Data_Acquisition_Report.* and Presentation.*: acquisition report and review deck.
- Dockerfile/docker-compose.yml and frontend Dockerfile: container deployment definitions.
- tests/: unit tests for AQI, API, data building, model evaluation, and source normalization.

DATA SOURCES AND HOW THEY ARE USED

1. OpenCity's Delhi hourly air-quality archive is the primary historical CPCB/DPCC station source. Documentation records 39 station
2. CPCB live snapshot via data.gov.in is intended for official station metadata/current snapshots. The connector requires an API ke
3. Open-Meteo Historical Weather supplies historical meteorological predictors (temperature, humidity, precipitation, wind, pressur
4. Open-Meteo Air Quality/CAMS supplies regional atmospheric-model fields. These are auxiliary estimates, not station-monitor measu
5. IIT Delhi AirDelhi supplies mobile-sensor/spatial benchmark data and a fine grid. It is normalized separately; the project's pol
6. NASA FIRMS provides satellite active-fire detections for Delhi and upwind regions. Fire detections are event proxies, not direct
7. OpenAQ supplies additional catalog, latest, and hourly measurements for cross-checking/backup. Credentials are external to the r
8. Kaggle and Hugging Face mirrors are for comparison and schema/provenance checks unless audited; they are not automatically merge
9. WAQI is the live two-station pilot source. Its PM2.5 individual index uses a different scale/meaning from CPCB concentration and

The documented data policy keeps source-specific files and manifests, records hashes, validates units/times/station identity, and avoids treating every downloaded dataset as ground truth. API keys are expected in environment variables or an ignored local .env file, never committed.

DATA PIPELINE

A. Acquisition: scripts/download_datasets.py and specialized scripts download OpenCity/CPCB references, OpenAQ, FIRMS, Kaggle, etc. Downloaders use resumable batches/manifests where implemented. External API credentials are required for some sources.

B. Normalization: source-specific normalize_* scripts harmonize timestamps, units, pollutant columns, duplicate handling, and station identifiers. Raw data remains separate. AirDelhi and the older AQI-only CPCB archive have separate outputs and are not silently mixed into concentration targets.

C. Auditing: analyze_cpcb_panel.py records station coverage, missingness, duplicates, temporal extent, network availability, and conservative coordinate matches. The experiment ledger reports 38 of 39 stations matched to live CPCB coordinates; site_106 (IGI Airport) remains unmatched rather than receiving a guessed location.

D. Feature building: build_multistation_dataset.py creates a station-hour panel with current and lagged pollutant observations, rolling summaries, calendar/time features, station metadata/identity, and nearest-neighbour summaries. Targets are direct PM2.5 values 1/3/6/12/24 hours ahead. build_unified_dataset.py extends features with aligned weather/CAMS/FIRMS sources. Dataset manifests record feature columns and provenance.

E. Model training/evaluation: baseline and challenger scripts train models on chronological data. Reported fixed split: training before 2025-07-01, validation 2025-07-01 to 2025-09-30, held-out test from 2025-10-01. Rolling quarterly backtests are also described. Test-tail results were already used in some later diagnostics; the ledger explicitly labels those follow-up experiments as internal historical results rather than untouched external benchmarks.

F. Serving bundle: build_serving_bundle.py packages compact station snapshots, recent history, model artifacts, and metadata for inference without copying research data into the service image. The README describes a roughly 48 MB local bundle and portable packaging. Those paths require generating/downloading the artifacts first.

FEATURE ENGINEERING AND LEAKAGE CONTROL

The historical feature design uses only information available at prediction time: current measurements, lagged measurements, rolling summaries, station identity/metadata, calendar cycles, nearest-station context, and auxiliary fields aligned to issue time. Direct forecast heads avoid recursively feeding an earlier prediction into the next forecast horizon. Code/documentation explicitly state that realized future pollution and future weather are not joined into issue-time features. Train/validation/test partitions are chronological; target times around boundaries are purged where needed. Sequence models use train-fitted normalization and missingness masks. These are sound controls, though the project still needs repeated future-period validation to establish robust real-world performance.

MODEL FAMILIES

- Persistence: last known PM2.5, a key baseline and a reasonable fallback.
- Global XGBoost: one gradient-boosted tree model per forecast horizon using pooled station observations and engineered features. I
- HistGradientBoosting: scikit-learn tree model used in some comparisons and the live index pilot.
- LSTM and TCN: 48-hour sequence challengers with missingness masks and station embeddings, trained at six hours in the documented
- Spatial GNN: each coordinate-matched station is a graph node linked to its six nearest neighbours. It incorporates recent station
- Multi-pollutant AQI: separate six-hour models for PM2.5, PM10, NO2, CO, and O3; their sub-indices are combined using the worst av
- Live index pilot: HistGradientBoosting models trained on historical CPCB PM2.5 transformed to a legacy US-EPA PM2.5 index proxy f

AQI CALCULATION AND SEMANTICS

The AQI module defines CPCB pollutant concentration breakpoints for PM2.5, PM10, NO2, SO2, O3, CO, and NH3. It rounds concentrations to integer precision, interpolates sub-indices within breakpoint bands, and uses the maximum available sub-index as overall AQI. It returns an explicit insufficient_for_overall result unless at least three pollutant sub-indices are available, including PM2.5 or PM10. For PM2.5-only input, the code returns a result labelled pm25_derived_proxy. This avoids silently presenting a single-pollutant index as official composite AQI.

The dashboard and API distinguish concentration (PM2.5, micrograms per cubic metre), CPCB AQI, and WAQI's individual PM2.5 index. This scale distinction is important: values from the live pilot cannot be interpreted as concentrations or directly compared with CPCB composite AQI scores.

REPORTED RESULTS: HISTORICAL CPCB PANEL

Panel audit (documented in EXPERIMENTS.md):

- 39 station files and 684,216 hourly rows.
- Coverage from 2024-01-01 00:00 UTC to 2025-12-31 23:00 UTC.
- Mean network hourly PM2.5 availability: 93.20%.
- No station below 60% structural hourly coverage.
- 38/39 station names matched to live CPCB coordinates; site_106 remains unmatched.

One-hour global XGBoost test (October through December 2025):

Metric	Persistence	Global XGBoost
MAE (micrograms/m3)	22.692	18.527

RMSE 37.143 29.853

R-squared 0.891 0.929

The ledger reports an 18.3% aggregate MAE improvement over persistence. It also reports a station regression: site_113 MAE increased from 7.956 to 12.006. Therefore the ledger did not promote the model on the basis of this one split alone.

Rolling-origin one-hour backtest (five expanding quarterly test periods):

- Mean fold MAE: persistence 16.383, XGBoost 14.466 (11.7% lower for XGBoost).
- XGBoost improved 176 of 195 station-fold slices and regressed on 19.
- Largest cited regression remained site_113 in the latest high-pollution quarter.

Six-hour sequence-model same-cohort comparison (6,708 test rows):

Model	MAE	RMSE
Persistence	66.812	94.043
Global XGBoost	42.814	63.032

LSTM 45.195 69.235

TCN 45.197 68.380

XGBoost had about 5.3% lower MAE than the sequence models in this comparison.

Spatial GNN comparison against final XGBoost and persistence:

Horizon	Rows	Persistence MAE	GNN MAE	XGBoost MAE
1h	80,224	22.879	21.802	18.887
3h	79,664	48.141	37.979	35.339
6h	79,254	70.650	49.393	44.246
12h	78,780	85.446	58.956	50.901
24h	78,257	58.672	61.145	56.117

GNN improves on persistence through 12 hours but loses to XGBoost at every horizon and loses to persistence at 24 hours. Reported A

Six-hour source/feature diagnostic (sampled training rows, full validation/test rows):

Features/model	Validation MAE	Test MAE
CPCB sensor/time/neighbour, XGBoost	11.834	47.930
+ weather, XGBoost	12.117	45.618
+ weather + CAMS, XGBoost	12.291	44.547
+ FIRMS (all sources), XGBoost	12.906	44.283
All sources, HistGradientBoosting	not reported	44.587
Validation-selected XGBoost/HGB blend	not reported	44.283

The validation summer period was easier than the polluted October–December test tail. Weather/CAMS/FIRMS reduce the six-hour test M

Six-hour five-pollutant subset AQI estimate:

- Same reported 81,074-row test cohort with PM2.5 available.
- Five-pollutant subset AQI: MAE 47.10, category accuracy 62.10%.
- PM2.5-only proxy: MAE 51.68, category accuracy 57.77%.
- AQI persistence: MAE 75.97, category accuracy 47.18%.
- Improvement over the PM2.5-only proxy: 4.58 MAE points and 4.33 percentage points category accuracy.

Caveats: SO2 and NH3 are omitted, the estimate uses an incomplete set of pollutants, and the October–December 2025 test tail was al

REPORTED RESULTS: WAQI LIVE PILOT

The tracked evaluation.json gives the training period ending 2025-07-01, validation July–September 2025, and test starting 2025-10-01. There are two sites. It predicts an individual PM2.5 index proxy (US-EPA scale). Test MAE in index points:

Horizon	History/spatial model	Single-observation incumbent	Weather candidate
1h	19.334	20.209	19.194
3h	34.590	37.296	33.117
6h	42.583	47.630	39.635
12h	48.179	51.489	44.774
24h	50.691	52.407	50.024

Weather models are promoted only if validation MAE improves and test MAE improves by at least 1%. The recorded gate passes for 12h

API, WEB APPLICATION, AND DEPLOYMENT

FastAPI (src/delhi_aircast/api.py) provides:

- GET /health: artifact/data availability and configured live key status.
- POST /aqi: CPCB pollutant sub-indices/AQI, or explicit PM2.5-only proxy.
- GET /stations and /stations/forecast: station catalogue/current snapshot and historical horizon forecasts.
- GET /forecast/{station_id}, /forecast/{station_id}/path: per-station forecasts and paths.
- GET /stations/{station_id}/history: recent historical PM2.5 observations.
- GET /live/stations/forecast: two-site WAQI index forecast, cache/refresh status, age, feature mode, and attribution.

The offline serving path loads final XGBoost artifacts first, falling back to multi-station artifacts. Six-hour forecasts use pollutant models only when all required subset artifacts are available; otherwise they report a PM2.5 proxy. The stored data is explicitly marked historical/offline. It may not be newer than the deployment bundle.

The web frontend is a Next.js/React TypeScript app with a station map, forecast horizons, PM2.5 history chart, AQI labels, and a separate live pilot panel. Requests are proxied through the frontend to the API. The repo also includes a Streamlit research dashboard for locally available feature/model bundles. Docker Compose runs the API and web service; a read-only data bundle is mounted into the API, and live collection history is stored separately. The API's live route calls WAQI from the server, so the token is not sent to the browser. README lists hosted demo endpoints on Vercel and Render; this inspection did not call or verify those external endpoints.

WHAT IS PRESENT IN THIS CHECKOUT

- Complete source code, docs, tests, figures, Word/PDF report, slide deck, and model strategy/experiment ledger.
- About 27 MB total project checkout size at inspection time.
- Tracked WAQI pilot evaluation.json and 15 compact joblib model files (history/spatial, incumbent, and weather candidates across f
- CPCB reference PDFs.

WHAT IS NOT PRESENT

- The large raw and processed data tree (data/raw, data/processed, data/runs). These are intentionally excluded from Git. The check
- The final historical XGBoost model weights described in the reports are published separately and/or generated locally; they are n
- A WAQI_API_KEY environment value was not checked or required for this report; live API behavior was not exercised.
- No full-stack tests/builds were run during this exploration. Existing test files indicate intended coverage but do not prove the

STRENGTHS

1. Clear separation between reference-station data, regional models, mobile-sensor grids, community datasets, and live index feed.
2. Explicit point-in-time feature design and chronological evaluation rather than random train/test splits.
3. Persistence comparisons and negative results are recorded; neural and graph methods were not chosen just for novelty.
4. AQI logic distinguishes official composite sufficiency from a single-pollutant proxy.
5. Provenance manifests, source hashes, resumable download patterns, tests, and reproducibility commands are built into the workflo
6. Separation of offline historical forecasts from a live pilot reduces the risk of presenting stale historical artifacts as live s

LIMITATIONS AND RISKS

1. Spatial granularity is station-level. A schematic/map visualization does not imply validated forecasts at every neighbourhood co
2. Core forecast data and XGBoost weights are absent from this checkout. Reported results cannot be independently rerun from this f
3. Evaluation periods are limited. Much evidence uses 2024-2025 data and a fixed Oct-Dec 2025 tail; later diagnostics reuse that ta
4. Site-specific errors matter. Aggregate MAE can hide regressions such as site_113; station, season, and pollution-event slices sh
5. Model quality worsens with horizon and differs by season; summer validation substantially underestimates the difficulty of winte
6. The GNN and sequence challengers have not beaten XGBoost on the cited cohorts. Their additional complexity is not currently just
7. Uncertainty intervals are called for in the strategy but not demonstrated in the serving contract/results reviewed here.
8. The AQI estimate is incomplete when some pollutants are missing. The six-hour five-pollutant subset omits SO2/NH3 and must remai
9. The live pilot covers two sites only, depends on WAQI availability/key and observation freshness, and requires a collector to bu
10. The live pilot's WAQI index scale differs from CPCB AQI and concentration units. It is explicitly not a health advisory.
11. Deployment/demo URLs are stated in README but were not tested in this inspection; no claim is made about current uptime.

RECOMMENDED NEXT STEPS

1. Rebuild and checksum the raw/processed data bundle from manifests; verify current source availability and licenses.
2. Re-run the full panel audit and all chronological/rolling-origin evaluations from a clean bundle, preserving immutable metric ou
3. Add a genuinely untouched later-period test and evaluate multiple winter/post-monsoon seasons.
4. Publish station-level and event-specific scorecards, including underprediction during severe episodes, category confusion, inter
5. Resolve site_113 regressions with station calibration or robust feature treatment; do not promote on aggregate metrics alone.
6. Implement and calibrate prediction intervals before presenting probabilistic forecasts.
7. Complete official-AQI pollutant coverage or keep every incomplete result clearly labelled as a proxy/subset.
8. Verify live collector uptime, persistent storage, key rotation, upstream failure behavior, and current WAQI performance on a pro
9. Add end-to-end checks for missing bundles, stale live feeds, unavailable weather, and mismatches between artifact feature schema
10. Keep the product wording precise: historical station forecast vs live two-site index pilot; concentration vs AQI; station locat

REPRODUCIBILITY COMMANDS (FROM PROJECT README)

From the project root, after obtaining its Python/Node prerequisites and data:

- uv run python scripts/download_datasets.py --source all
- uv run python scripts/normalize_airdelhi.py
- uv run python scripts/analyze_cpcb_panel.py
- uv run python scripts/build_multistation_dataset.py
- uv run python scripts/build_unified_dataset.py
- uv run python scripts/train_multistation_baseline.py --horizon 1
- uv run python scripts/compare_model_strategies.py --horizons 1 6 24 --n-jobs 4
- uv run python scripts/train_spatial_gnn.py --horizon 6 --epochs 20 --lr 0.001 --patience 5
- uv run python scripts/train_multipollutant_aqi.py --horizon 6 --run-dir data/runs/multipollutant_aqi_final
- uv run python scripts/build_serving_bundle.py --output .local-serving
- uv run uvicorn delhi_aircast.api:app --reload --host 127.0.0.1 --port 8000
- npm --prefix aircast-web ci
- npm --prefix aircast-web run dev

CONCLUSION

Delhi AirCast is a substantial academic research and demonstration system rather than a single model file. It covers collection and normalization, station-panel audits, leakage-aware feature construction, several model families, AQI computation, API/web serving, and deployment packaging. Its strongest documented result is the practical global tree model, which generally beats persistence and the evaluated sequence/graph challengers on the reported cohorts. Its main caveats are reproducibility from this checkout (data/model bundle omitted), finite historical evaluation, station-level regressions, incomplete pollutant coverage, and the narrow scope and distinct scale of the WAQI live pilot. Any presentation should preserve those distinctions and avoid describing historical artifacts as live neighbourhood forecasts.

PRIMARY PROJECT REFERENCES

- README.md
- problem.md
- DATA_SOURCES.md
- MODEL_STRATEGY.md
- EXPERIMENTS.md
- src/delhi_aircast/aqi.py
- src/delhi_aircast/api.py
- aircast-web/app/page.tsx
- models/waqi_live_index/evaluation.json
- Delhi_AirCast_PS13_Project_Report.pdf
- Delhi_AirCast_PS13_Data_Acquisition_Report.pdf
- Delhi_AirCast_PS13_Presentation.pdf

END OF REPORT